PACKAGE 07 / VOLUME 06

Module 26: Collision detection and physical simulation

Four expanded lessons • Project and review

UPPER UNDERGRADUATE
Modules 26-30 · Lessons 26.1-30.4 · Gate F

Edition: September 28, 2026

Original expanded instruction, worked examples, quizzes, implementation contracts and evidence-based review. The learner performs the projects and platform/human validation. This self-study curriculum is not an accredited degree.

Contents

Module 26: Collision detection and physical simulation 4

Lesson 26.1: Broad phase, narrow phase and geometric robustness . . . 5

Lesson 26.2: Rigid bodies, angular motion and collision impulses . . . 8

Lesson 26.3: Constraints, iterative solvers and continuous collision detection . . . 11

Lesson 26.4: Physics testing, determinism and engine integration . . . 14

Module 26 project . . . 16

Module 26 review . . . 17

Reference Directory 18

S01: Collision documentation . . . 18

S02: Simulation documentation . . . 18

S03: glTF tutorial: Skins . . . 18

S04: glTF 2.0 specification . . . 18

S05: Writing a callback function . . . 18

S06: SDL_AudioStreamCallback . . . 18

S07: Xbox Accessibility Guidelines . . . 18

S08: CS188: Informed search . . . 19

S09: Introduction . . . 19

S10: CS188: Minimax . . . 19

S11: Automatic differentiation with torch.autograd . . . 19

S12: Networked movement in the Character Movement Component . . . 19

S13: Snapshot Interpolation . . . 19

S14: GGPO SDK repository 19

S15: Transaction documentation 20

S16: Atomic Commit in SQLite 20

S17: os.replace documentation 20

S18: zipfile documentation 20

S19: Fix Your Timestep! 20

S23: XAG 103: Additional channels for audio and visual cues 20

S24: Deterministic Lockstep 20

S25: XAG 107: Input 20

S26: XAG 101: Text display 21

Original program reading IDs 21

Module 26: Collision detection and physical simulation

Prerequisites: Modules 07-10, 14 and 24.

Original reading IDs: R28, R19, R23. See the reference directory.

Module goal: Build a deterministic test harness for a small physics scene, compare your simple implementation with a maintained physics library, and record precisely which physical and numerical assumptions differ.

Lesson 26.1: Broad phase, narrow phase and geometric robustness

Learning goal: Separate possible collisions from confirmed contacts.

The engineering problem

A projectile passes through a door only when the camera looks away. Before changing an epsilon, determine which subsystem owns collision membership and whether its update depends on rendering. Collision is simulation work. A visibility optimization must not remove an object from the collision world. Write the contract first: positions are meters, axes are right-handed in the simulation, AABBs are closed intervals, and normals point from body A toward body B.

Broad phase is a conservative filter

For n objects, exhaustive testing considers n(n-1)/2 unordered pairs. With 100 objects this is 4,950. Keep that implementation as an independent oracle. An optimized broad phase is correct when every truly overlapping pair is a candidate. It may return extra candidates. Comparing candidate sets for equality is therefore the wrong test; compare true contacts against the candidate superset, then compare narrow-phase results.

A uniform grid inserts each object's complete bounding box into every cell it overlaps. Compute a cell coordinate with floor(x/cell_size), not integer truncation toward zero. An object crossing the origin reveals the difference. Insert a closed box's maximum boundary conservatively even when it lies exactly on a cell edge. Emit canonical pairs (min_id, max_id), deduplicate them, and sort before the solver consumes them. Inserting only the center is not conservative for objects larger than a cell.

Grid performance depends on occupancy and object size. A crowded cell can still contain almost every pair. A huge object can occupy thousands of cells. Keep membership and pair counts in the profiler; a data structure name does not imply a speedup. A dynamic AABB tree is an alternative for heterogeneous sizes, while sweep-and-prune can exploit coherence along an axis. Box2D's collision documentation provides maintained shape queries and a dynamic-tree comparison target [S01].

Narrow phase and the meaning of contact

For circles, let d = center_B - center_A and R = radius_A + radius_B. Overlap under our closed-set policy means dot(d,d) <= R*R. Only after this decision do we need a square root for distance and a normalized contact normal. If the centers coincide, dividing by distance is invalid. Choose a deterministic fallback normal, such as the previous valid normal or a stable axis derived from IDs, and document it.

For AABBs, reject the pair when either axis separates the intervals. For convex polygons, the separating-axis theorem motivates projecting both polygons onto candidate edge normals: a separating interval proves no overlap. Absence of separation is not yet a complete contact manifold. Concave meshes need decomposition or a different representation. Do not present

the circle lab as a finished general-purpose collision library.

Worked trace: a cell boundary that breaks naive code

Let cell size be 1 m. Circle A has center (-0.1, 0), radius 0.2 m; circle B has center (0.15, 0), radius 0.1 m. Their center distance is 0.25 m and their radius sum is 0.3 m, so they overlap. A's x bounds are [-0.3, 0.1], spanning cells -1 and 0. B's bounds are [0.05, 0.25], in cell 0. Both also span y cells -1 and 0. A center-only or incorrectly truncated implementation can conceal a missing-cell defect in other placements. The robust implementation inserts full bounds and deduplicates shared-cell pairs.

Translate this entire scene by exactly 10 m and repeat. Geometry should be unchanged within the declared numeric tolerance. Repeat with a coincident center, an exact tangent and a radius near the minimum supported scale. These are different questions: boundary policy, degeneracy policy and scale policy must not be collapsed into one unexplained constant.

Implementation and debugging discipline

```
validate finite coordinates and positive cell size
for each stable body ID:
    compute shape AABB
    insert ID into every covered cell
for each occupied cell:
    emit all distinct ID pairs in canonical order
sort and unique candidates
run narrow phase; record contact IDs and normals
```

In a debug capture, retain bounds, cell memberships, candidate IDs and final contacts for the first missed pair. If a collision disappears, this trace identifies whether membership, filtering, geometry or lifetime failed. Reject invalid geometry before indexing it. Bound object size and cell coverage before allocating storage for user-authored scenes.

Deliberate practice

First solve the pair calculation on paper. Then implement the exercise kernel without reading its reference. The full lab adds a seeded randomized oracle comparison and an engine-owned spatial index. Measure uniform, clustered and one-giant-object scenes. A correct but slower grid is an acceptable finding; report it rather than choosing a convenient scene after seeing results.

Original worked checkpoint

For 100 objects, checking every unordered pair requires 100 x 99 / 2 = 4,950 pair tests. A broad phase helps only if its maintenance cost and candidate count are favorable for the scene.

Full practical assignment

Implement a grid broad phase and circle or box narrow phase. Pass: no true pairs are omitted compared with brute force over randomized scenes, and boundary-touching behavior is documented.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Conservative grid and circle contacts**. Run `starter/python/demos/lesson_26_1.py`; implement `starter/python/exercises/lesson_26_1.py`.

Assigned study and native transfer

R28 : collision module, shapes and broad-phase concepts; R19 : geometry and collision.

Read S01: shape primitives, geometric queries, contact manifolds and dynamic tree. Read the original R19 geometry material as a mathematical comparison. Record the exact library release used for any comparison.

Engine transfer: Port the broad-phase ownership and pair-order contract into the Volume 5 engine. Keep the brute-force oracle in a debug/headless target, independent of the grid implementation.

Check your understanding

1. May a correct broad phase return false positives?
2. How many unordered pairs exist for 100 objects?
3. Why document contact-normal orientation?
4. With 250 bodies, how many exhaustive unordered pairs are possible? What relationship must hold between true-contact and candidate sets?
5. A circle spans four cells but its center occupies one. Is center-only insertion sufficient? Give a repair and a duplicate-pair policy.

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 26.2: Rigid bodies, angular motion and collision impulses

Learning goal: Use conservation laws with explicit approximations.

State, units and approximation

A rigid body has a center-of-mass position, orientation, linear velocity, angular velocity, mass and inertia. A two-dimensional body has scalar angular velocity around the perpendicular axis. In three dimensions, angular velocity is a vector and inertia is a tensor whose frame matters. This lesson derives a 2D contact; the same conceptual separation between translation and rotation carries forward to 3D.

Force has units kg*m/s^2 and changes momentum over time. Impulse has units kg*m/s and changes velocity immediately in the model. Applying a force as an impulse every tick makes behavior depend on tick rate. Conversely, multiplying an already integrated impulse by dt weakens it incorrectly. Keep distinct API names: apply_force and apply_impulse. Static bodies use zero inverse mass and zero inverse inertia, not a floating-point infinity.

Deriving the contact impulse

Choose a unit normal n from A to B. Let r_A and r_B run from the bodies' centers of mass to the contact. The relative contact-point velocity is v_B + omega_B cross r_B - v_A - omega_A cross r_A. Its projection v_n on n is negative when the bodies approach under this convention. An impulse +j*n goes to B and -j*n to A. For a frictionless instantaneous collision:

```
k = inv_mass_A + inv_mass_B
    + (cross(r_A,n))^2 * inv_inertia_A
    + (cross(r_B,n))^2 * inv_inertia_B
j = -(1 + restitution) * v_n / k
```

Apply this only to an approaching contact with positive effective inverse mass k. For a separating contact, a positive bounce impulse would manufacture energy. The rotational terms explain why the same contact force can produce different linear outcomes at different lever arms. A normal is a direction, not a penetration distance; normalize it before using these equations.

For a center-of-mass one-dimensional collision, the rotational terms disappear. Equal masses m = 2 kg start at v_A = 3 m/s and v_B = 0. With restitution e = 0.5, relative velocity is -3 m/s, k = 1 kg^-1, and j = 4.5 N*s. The final velocities are 0.75 and 2.25 m/s. Momentum remains 6 kg*m/s. Initial kinetic energy is 9 J and final energy is 5.625 J. The energy loss is expected in this inelastic model, not a failed conservation test.

Worked rotational impulse

An isolated 2 kg body receives J = (6,0) N*s at offset r = (0,0.5) m. Its linear velocity changes by (3,0) m/s. Angular impulse is cross(r,J) = -3 kg*m^2/s. With inertia 0.25 kg*m^2, angular velocity changes by -12 rad/s. The sign follows the stated right-handed convention. If the application and renderer disagree about vertical-axis direction, the same physical clockwise motion may appear with an inverted screen angle; fix the transform boundary rather than negating impulses opportunistically.

Friction, resting contacts and energy

A tangential impulse opposes relative sliding. A simple Coulomb constraint bounds its magnitude by mu times the accumulated normal impulse. In an iterative solver, clamp the accumulated tangential impulse, then apply only the increment. Clamping each increment independently can exceed the intended total friction bound. Static and kinetic friction are richer than this compact approximation, especially around transitions and rolling contact.

Restitution is a collision-event model, not a demand that a resting stack bounce every tick. Many engines use a velocity threshold or solver-specific treatment. Study the selected library's contact and material behavior rather than assuming a universal parameter mapping [S02]. Gravity and the floor supply external forces and impulses, so momentum of only the falling body is not conserved. Select an isolated two-body scene when testing total momentum.

Failure analysis and integration

Check finite state, nonnegative inverse masses, supported restitution and inertia before applying impulses. A static-static contact has zero effective inverse mass and must not divide by zero. Distinguish a dynamic body temporarily at rest from a truly static body. Do not set inverse mass to zero merely because its current velocity is small.

The kernel lab handles center-of-mass velocity updates. The full lab must add the off-center case above, momentum and energy tables, an immovable wall, and tests across at least two fixed timesteps for sustained forces. Integrate through stable body handles. Queue creation and destruction outside solver iteration so callbacks cannot invalidate the contact list.

Original worked checkpoint

A 2 kg body receiving a 6 N*s impulse through its center changes velocity by 3 m/s. The same impulse away from its center also changes angular momentum.

Full practical assignment

Implement center-of-mass impulses and a simple rotational case. Pass: momentum is conserved in isolated tests within tolerance, static bodies remain fixed, and energy behavior matches the chosen restitution model.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **One-dimensional collision impulse**. Run `starter/python/demos/lesson_26_2.py`; implement `starter/python/exercises/lesson_26_2.py`.

Assigned study and native transfer

R28 : rigid bodies, contacts and simulation; R19 : physical simulation.

Read S02: rigid bodies, mass data, forces and impulses, friction and restitution. Re-derive the center-contact impulse without copying the supplied equation.

Engine transfer: Implement separate force and impulse commands in your engine. Add tests proving that the renderer cannot directly overwrite the physics-owned transform of a dynamic body.

Check your understanding

1. What velocity change follows from 6 N*s applied to 2 kg?
2. Why can an off-center impulse cause rotation?
3. Why use zero inverse mass for a static body?
4. Two 2 kg bodies have velocities 3 and 0 m/s and restitution 0.5. Compute impulse, final velocities and total final kinetic energy.
5. A 6 N*s horizontal impulse acts 0.5 m above the center of a body with inertia 0.25 kg*m^2. What angular-velocity change results?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 26.3: Constraints, iterative solvers and continuous collision detection

Learning goal: Explain where discrete simulation fails and how solvers control error.

Two distinct sources of error

A projectile can miss a wall because collision was sampled only at endpoints. A chain can stretch because its constraint solver stopped before convergence. Increasing solver iterations addresses the second problem but does not necessarily discover the missing collision in the first. Diagnose detection, integration and constraint solving independently.

A distance constraint between particles A and B has residual C = length(x_B-x_A) - rest_length. A residual is a measured violation, not a visual impression. For a nonzero separation direction n and inverse masses w_A, w_B, a position-projection correction can distribute error in proportion to inverse mass:

```
x_A += n * C * w_A / (w_A + w_B)
x_B -= n * C * w_B / (w_A + w_B)
```

Skip an immovable pair and handle coincident points explicitly. A projection method is useful for learning convergence but is not identical to a velocity-level rigid-body solver. If corrected positions are converted to velocities, corrections can inject energy. Name the algorithm and measure the quantities it actually promises.

Iterative contact solving

A sequential impulse solver visits constraints, updates velocities locally, and repeats. Contact order affects intermediate results; warm starting reuses prior accumulated impulses to begin closer to a solution. For unilateral normal contact, accumulated normal impulse must remain nonnegative. Compute a proposed change, clamp the accumulated result, then apply only the difference. Otherwise the solver can pull bodies together with an attractive normal force.

Track the maximum and RMS residual across a three-link chain. Record initial positions, timestep, solver iteration count, ordering, gravity and tolerance. Compare 1, 2, 4, 8 and 16 iterations, then halve dt. A monotonic trend in one scene is evidence about that setup, not a guarantee for every contact graph. Box2D's documented sub-step count is not a drop-in synonym for every engine's solver-iteration parameter [S02].

Sweeping through time

At 30 m/s and 60 Hz, a projectile travels 0.5 m each tick. A 0.1 m wall can lie entirely between the starting and ending positions. For a point moving along x with constant velocity v toward a static slab [lo, hi], compute t0 = (lo-x)/v and t1 = (hi-x)/v, sort them, and intersect [t0,t1] with [0,dt]. Initial overlap has a declared hit time of zero in this teaching kernel. Zero velocity outside the slab means no hit. Zero velocity inside means initial overlap.

For an axis-aligned moving box, expand the obstacle by the box's half extents and sweep its center. In several dimensions, intersect the time intervals from each slab. For relative linear

motion of two objects, subtract velocities first. Rotation complicates swept bounds and contact geometry; a point-versus-slab result is not general rotational CCD. Library cast APIs may deliberately treat initial overlap differently, which must be recorded in the comparison [S01].

Worked time of impact

A point starts at x=0 with v=30 m/s; the wall occupies [0.24,0.26] m. Entry occurs at 0.24/30 = 0.008 s and exit at about 0.008667 s. Both occur before the 1/60 s tick ends. The endpoint at 0.5 m is outside the wall, but a sweep detects the entry. With a radius of 0.02 m, expand the wall to [0.22,0.28], so entry is about 0.007333 s.

After finding the first impact, advance to the impact time, solve the collision, and account for the remaining interval according to your policy. Repeated impacts can produce a near-zero-time loop. Bound the number of impacts per tick and specify a conservative fallback, such as stopping the fast body and emitting a diagnostic. Never let a malformed level create unbounded simulation work.

A disciplined experiment

Implement the swept kernel, then a chain residual experiment. The workbook requires both; a passing slab test does not complete the lesson. Keep raw residual tables rather than only a screenshot. Include a static pair, coincident anchors, a separating projectile, an initial overlap, and an impact exactly at dt. Measure whether more iterations or smaller dt delivers the better error/cost tradeoff in your chosen scene.

For the engine, CCD belongs to selected bodies and motion regimes. It is not a switch that cures all physics failures. Keep a record of which objects use sweeps, which use discrete contacts, and how impact events are scheduled relative to the presentation layer.

Original worked checkpoint

An object moving 30 m/s travels 0.5 m during a 1/60 s step. A wall much thinner than that can be crossed between sampled positions without either endpoint overlapping.

Full practical assignment

Create a fast-projectile tunneling test and a short chain of constraints. Pass: a continuous or swept test catches the projectile, and a plot or table shows constraint error versus iteration count and timestep.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Swept point versus one-dimensional slab**. Run `starter/python/demos/lesson_26_3.py`; implement `starter/python/exercises/lesson_26_3.py`.

Assigned study and native transfer

R28 : joints, solver settings and continuous collision; R19 : numerical simulation.

Read S01: ray casts, shape casts and time of impact. Read S02: stepping and joints. Read S19 for fixed-step accumulator behavior. Do not conflate substeps, solver iterations and render frames.

Engine transfer: Add a debug CCD flag, a per-tick impact cap and residual telemetry to your engine. Demonstrate that a dense contact scene cannot make one tick loop without a bound.

Check your understanding

1. How far does 30 m/s travel in 1/60 second?
2. Why can increasing solver iterations help?
3. Does CCD remove every numerical simulation problem?
4. A point starts at 0 m and moves at 30 m/s toward the slab [0.24,0.26] m during a 1/60 s tick. What is the entry time, and why does an endpoint test miss it?
5. A chain becomes less stretchy when solver iterations increase. Does this prove conservation of energy or solve projectile tunneling?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Lesson 26.4: Physics testing, determinism and engine integration

Learning goal: Validate simulation with invariants and adversarial scenes.

Define exactly what repeatable means

Repeatability is a claim with a scope: same executable and machine; same compiler settings on different machines; or bitwise agreement across named platforms. A visually similar replay is weaker than an identical serialized state. A hash proves agreement only for the bytes you included. Excluding the random generator, an AI cooldown or a spawn counter can make a misleadingly stable hash.

Write a state schema with stable entity IDs, transforms, velocities, gameplay fields, random state and pending deterministic commands. Exclude pointer addresses, padding, wall-clock timestamps and rendering interpolation. Define field order, integer widths, byte order and floating-point representation. Reject NaNs before hashing because NaNs have unusual comparison behavior and multiple bit patterns. Decide how negative zero is treated. Hashes localize differences; they do not make arithmetic deterministic.

The simulation boundary

Let S_t be the state before tick t. The transition consumes the input bundle I_t and produces S_(t+1) plus an event list E_t. Use an integer tick as the authoritative time index. An accumulator can run zero or more fixed steps per display frame, retaining a previous/current transform pair for rendering interpolation. Cap catch-up work and disclose the overload policy; silently discarding time changes behavior and must not be disguised as successful real-time operation [S19].

Stable iteration order matters when operations are not associative. Three floating-point values can sum differently under different grouping. Thread scheduling, unordered-container traversal and compiler contraction can therefore matter. The current Box2D simulation documentation describes explicit determinism strategies, including creation ordering and compiler assumptions, and warns that deterministic library behavior does not make an entire application deterministic [S02]. Do not repeat the outdated blanket claim that all physics-library cross-platform determinism is impossible.

Analytic checks expose integration error

For constant acceleration a, an exact one-dimensional reference is x(t)=x0+v0*t*+*0.5*a*t*t and v(t)=v0+a*t. Semi-implicit Euler updates velocity first and then position. After N equal steps of size h, position is x0+v0*N*h+*0.5*a*h*h*N*(N+1). Its position error against the exact solution is 0.5*a*N*h*h, or 0.5*a*t*h. At a=-10 m/s^2, t=1 s and h=1/60 s, error is about -0.083333 m. Halving h should roughly halve this error for this experiment. The expected error is not zero.

A force-free body should maintain velocity. An isolated elastic two-body collision should conserve momentum and kinetic energy within tolerance. A damped system should not be judged by energy conservation. An engine under gravity should not be judged by constant

kinetic energy. Write the physical assumptions beside every invariant.

First-divergence debugging

Run a fixed input tape twice and compare state hashes at every tick. Deliberately alter one input at tick 37. The first differing post-step state should be S_38 under the naming convention above. If only the final hash is stored, hundreds of later contacts obscure the cause. Save the pre-divergence state, the input bundle, candidate/contact order, random state and build metadata. Then compare decoded fields, not just two opaque digest strings.

A serialization round trip is a separate test. Restore a state, advance the same next input, and compare against the unrestored run. Restoration that omits a warm-start cache may preserve visible transforms but change the next tick. Whether such caches belong in a rollback snapshot depends on the engine and the exact determinism goal.

Engine lifetime and fault policy

Do not destroy physics bodies while iterating the library's event buffer unless the API explicitly permits it. Convert events to stable engine handles, validate generations and enqueue commands for a known safe phase. Renderer interpolation reads snapshots rather than mutable solver storage. Entity deletion should clear presentation ownership without allowing a recycled ID to receive an old collision sound.

On a non-finite value, record the first responsible tick and operation, then stop the test or isolate the body under a documented runtime policy. Replacing every NaN with zero can hide a persistent defect and produce impossible rewards or movement. The full lab combines analytic tests, repeatability, serialization and an adversarial scene suite. This is the evidence foundation for later rollback networking, not an optional debugging convenience.

Original worked checkpoint

A freely moving body with no force should maintain velocity in the ideal model. A falling body before collision can be checked against an analytic constant-acceleration solution, separating integration error from collision behavior.

Full practical assignment

Build a regression suite with replay hashes on one pinned environment, finite-value checks and analytic tests. Pass: failures identify the first divergent tick and reports distinguish same-machine repeatability from cross-platform determinism.

The complete procedure, failure matrix and acceptance evidence are in the corresponding LAB_WORKBOOK card. The small kernel is **Canonical replay and first divergence**. Run `starter/python/demos/lesson_26_4.py`; implement `starter/python/exercises/lesson_26_4.py`.

Assigned study and native transfer

R28 : simulation, determinism and limitations; revisit Modules 08 and 24.

Read S02: Determinism and simulation events. Read S19: fixed timestep and overload. Revisit original Modules 08 and 24 for numerical error and ownership.

Engine transfer: Publish a versioned engine state schema and first-divergence replay report. Classify the result as same-environment repeatability unless other named platform combinations were actually tested.

Check your understanding

1. Is same-machine repeatability proof of cross-platform determinism?
2. Why identify the first divergent tick?
3. What should happen when a simulation first creates a NaN?
4. Using semi-implicit Euler with acceleration -10 m/s^2, zero initial velocity, t=1 s and h=1/60 s, what is the position error against the analytic solution?
5. Two replay hashes agree, but an omitted AI cooldown differs. What has been proved, and what must change?

Attempt all five before opening the answer key. A passing quiz alone does not complete the lesson.

Module 26 project

Build a Physics Evidence Bench. Select a bounded scene containing two isolated colliding bodies, a static wall, one offset contact, a fast projectile and a short constrained chain. Keep scenes small enough to inspect every state transition. Use stable IDs, meters/seconds and a fixed update schedule. A graphical view is useful, but the headless state trace is the regression authority.

Integrate all four lessons: a conservative broad phase, narrow-phase contact convention, center/angular response, swept collision case and replay harness. Record the free-flight integrator separately from the contact solver. Run at least three timestep settings and several solver budgets without hiding energy injection or unresolved penetration. A limitation is acceptable when it is measured and bounded.

Compare at least one matched scene with a maintained library such as Box2D, using the exact library revision and settings. Do not substitute an online claim for running the comparison. The reference package includes a comparison template but does not download or execute that external library. No equality is promised when algorithms, skin widths or restitution thresholds differ.

Deliver scene files, seed/input tape, canonical state schema, invariant tests, first-divergence capture, raw measurements and a comparison report. Review asks: which invariants should hold here, what tolerances are numerical rather than geometric, and which differences are modeling choices?

Module 26 review

Answer these after the project work. They are additional to the 20 lesson questions in this module.

1. A projectile is missed only near cell boundaries. Which intermediate artifacts distinguish a broad-phase defect from a narrow-phase defect?
2. Why can an off-center contact require a smaller impulse than a center-only calculation for the same initial normal speed?
3. An engineer doubles solver iterations and calls it equivalent to halving the timestep. Explain the error.
4. What extra evidence is needed beyond matching two final-state hashes?
5. Your simple engine and a maintained library disagree on bounce height. What should be controlled before deciding either is wrong?

Reference Directory

S-prefixed entries are the focused primary-source companions consulted for this expansion, September 28, 2026. Numbers are stable identifiers; gaps are intentional. R-prefixed entries preserve the original program reading directory. External books, papers, documentation and library source are not bundled; the explanations, exercises and demonstration code in this package are original teaching material. Pin installed versions in your own evidence rather than treating a mutable documentation page as an installed build.

S01: Collision documentation

Box2D project. Shape queries, broad phase, dynamic tree and time-of-impact limitations.

Open primary source

S02: Simulation documentation

Box2D project. Stepping, contacts, solver settings and the documented determinism contract.

Open primary source

S03: glTF tutorial: Skins

Khronos Group. Joint transforms, inverse bind matrices and mesh-node correction.

Open primary source

S04: glTF 2.0 specification

Khronos Group. Normative asset, animation and skinning data contracts.

Open primary source

S05: Writing a callback function

PortAudio project. Callback constraints and real-time-safe execution discipline.

Open primary source

S06: SDL_AudioStreamCallback

SDL project. SDL3 audio stream callback contract; consult the API for the chosen integration.

Open primary source

S07: Xbox Accessibility Guidelines

Microsoft. Game accessibility design/testing guidance, not a legal-compliance certification.

Open primary source

S08: CS188: Informed search

University of California, Berkeley. A*, heuristics and search guarantees under stated assumptions.

Open primary source

S09: Introduction

BehaviorTree.CPP project. Behavior-tree concepts, statuses and implementation context.

Open primary source

S10: CS188: Minimax

University of California, Berkeley. Adversarial search and alpha-beta reasoning.

Open primary source

S11: Automatic differentiation with torch.autograd

PyTorch project. Gradient/parameter concepts; framework use is optional for the core synthetic exercise.

Open primary source

S12: Networked movement in the Character Movement Component

Epic Games. A maintained engine comparison for authority, prediction and correction.

Open primary source

S13: Snapshot Interpolation

Glenn Fiedler. Author’s technical explanation of interpolation, jitter and visual delay.

Open primary source

S14: GGPO SDK repository

GGPO authors. Primary implementation/documentation for rollback networking concepts.

Open primary source

S15: Transaction documentation

SQLite project. Transaction boundaries and behavior.

Open primary source

S16: Atomic Commit in SQLite

SQLite project. Atomicity assumptions and storage failure considerations.

Open primary source

S17: os.replace documentation

Python Software Foundation. Replacement semantics and platform/filesystem conditions.

Open primary source

S18: zipfile documentation

Python Software Foundation. Archive processing and extraction boundaries.

Open primary source

S19: Fix Your Timestep!

Glenn Fiedler. Simulation/render time separation and work-budget hazards.

Open primary source

S23: XAG 103: Additional channels for audio and visual cues

Microsoft. Critical information through more than one channel.

Open primary source

S24: Deterministic Lockstep

Glenn Fiedler. Input synchronization and deterministic simulation requirements.

Open primary source

S25: XAG 107: Input

Microsoft. Input/remapping accessibility considerations.

Open primary source

S26: XAG 101: Text display

Microsoft. Readable interface text and testing considerations.

Open primary source

Original program reading IDs

These inherited assignments remain part of the full study route. S entries above focus the expanded lesson tasks; they do not replace all external study.

R17: RFC 9293: Transmission Control Protocol — IETF / RFC Editor. Transport semantics; reference reading after networking foundations. Assigned source

R19: 15-462/662 Computer Graphics — Carnegie Mellon University. Rendering, geometry and animation course structure and exercises. Assigned source

R20: 15-466/666 Computer Game Programming — Carnegie Mellon University. Runtime systems and game-production programming. Assigned source

R22: SDL3 documentation — SDL project. Windowing, input, audio and platform services; do not mix SDL2 and SDL3 examples. Assigned source

R23: Physically Based Rendering, fourth edition — Matt Pharr, Wenzel Jakob, Greg Humphreys. Rendering mathematics and implementation; use chapter titles in lesson assignments. Assigned source

R26: Programming with C++ in Unreal Engine — Epic Games. Engine comparison after C++ foundations, not a beginner substitute. Assigned source

R28: Box2D documentation — Box2D project. Collision, constraints and simulation reference. Assigned source

R29: Xbox Accessibility Guidelines — Microsoft. Accessibility design and testing; not a legal-compliance certificate. Assigned source

R45: AddressSanitizer documentation — LLVM Project. Memory-error instrumentation, limitations and supported toolchains. Assigned source

R46: CS188 Introduction to Artificial Intelligence textbook — University of California, Berkeley. Search, games, decision processes, learning and probabilistic reasoning. Assigned source

R47: Dive into Deep Learning — Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola. Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions. Assigned source